Assignment 2 - Deep Q-Network

Diego HW2 | MLDS | Due November 11, 2026, 11:59 pm

GitHub username: Dorrie-ge

Repository: mlds-diego-hw2-dqn

Public source URL: https://github.com/Dorrie-ge/mlds-diego-hw2-dqn

My objective and results

I implemented DQN with a neural action-value function, experience replay and a separately synchronized target network for both required environments. I used gamma = 0.95 for CartPole-v1 and gamma = 0.99 for image-based ALE/MsPacman-v5 with nine actions. I base every result in this report on my retained experiment logs.

My greedy policy	Test episodes	Mean raw reward	Sample SD
CartPole-v1	500	500.00	0.00
ALE/MsPacman-v5	500	2752.08	798.01

How I reproduce and audit my work

I provide my source code, raw CSV/JSON, figures and reproduction commands in the public repository. I reproduce the deepdish environment with setup_deepdish.sh, select one GPU with CUDA_VISIBLE_DEVICES, and run bash run_deepdish.sh with GITHUB_USER and REPO_URL set. I describe individual commands and the Times New Roman font prerequisite in docs/REPRODUCING.md. I select checkpoints on validation seeds before inspecting the 500 final test episodes.

I ran training on mlds-deepdish1 with one NVIDIA GeForce GTX 1080 visible to PyTorch 2.5.1+cu118 and CUDA 11.8. I retain the execution log, hardware information and timestamps in artifacts/run/; I keep each game's exact configuration beside its result CSVs.

1. My DQN implementation

My learning rule and architectures

I sample replay tuples (s, a, r, s_next, terminated) and form y = r + gamma (1 - terminated) max_b Q_target(s_next, b). I minimize Huber loss between Q_online(s,a) and y, clip gradient norms to 10, and synchronize a detached target network at fixed intervals. I report ordinary DQN; I include an optional --double implementation for separate future ablations but did not enable it in these runs.

Setting	My CartPole model	My MsPacman model
Input / network	4 values; MLP 128-128 ReLU	4 x 88 x 80 pixels; CNN; dense 512
Convolutions	Not applicable	32/64/64 channels; 8x8/4, 4x4/2, 3x3/1
Gamma; Adam learning rate	0.95; 0.001	0.99; 0.0001
Replay capacity; warmup	50,000; 1,000 transitions	100,000; 20,000 transitions
Batch; update frequency	64; every 4 decision steps	64; every 4 decision steps
Target synchronization	Every 500 decision steps	Every 10,000 decision steps
Epsilon schedule	1 to .02 over 20,000 steps	1 to .01 over 1,000,000 steps
Training rewards	Raw survival rewards	Sign-clipped game rewards
Reported performance	Raw undiscounted rewards	Raw undiscounted game scores

How I process images and preserve the task

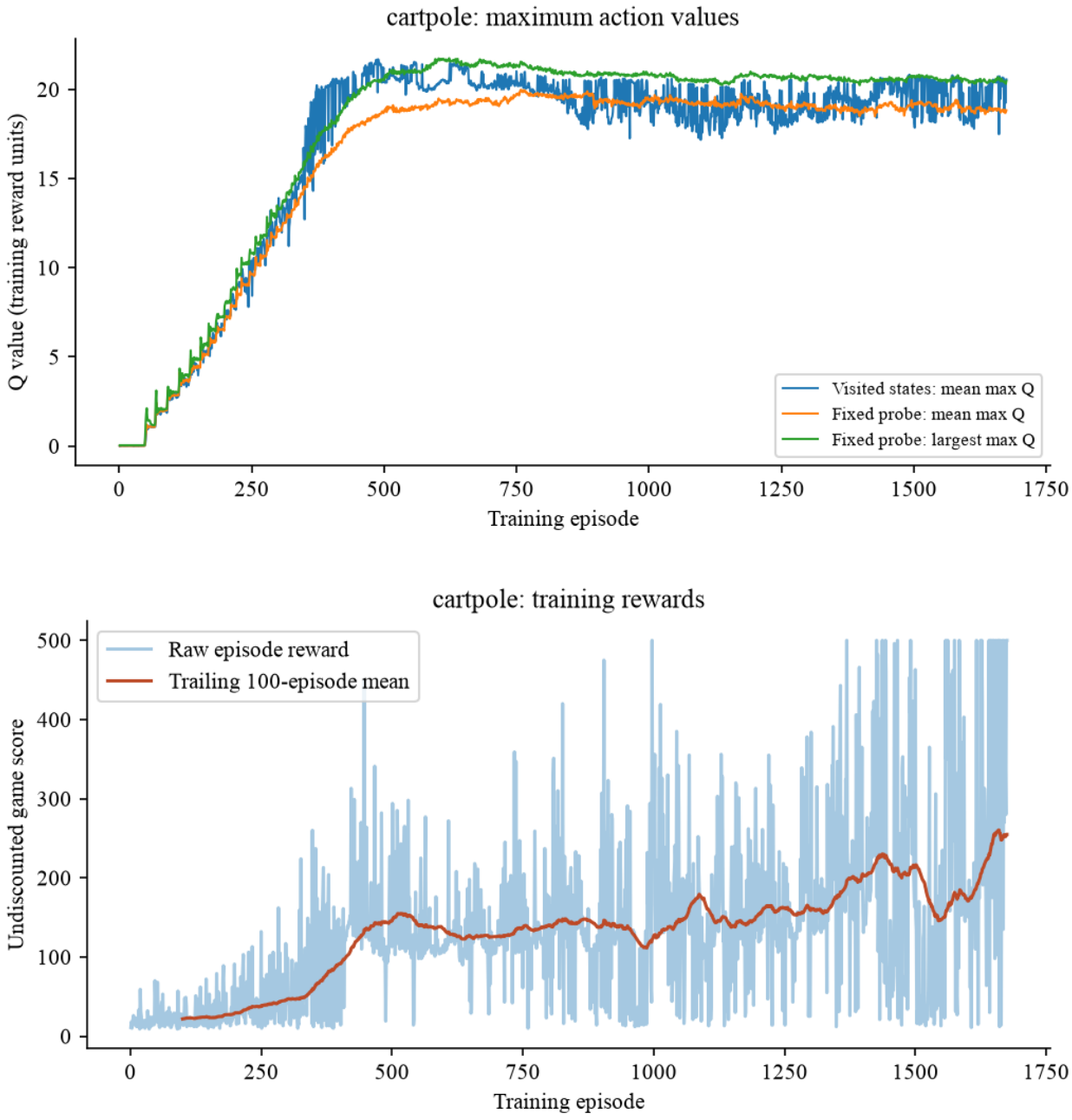
I use the assignment's crop and grayscale transform, including removal of pixels with RGB sum 448. I store uint8 intensities and center/scale them in the CNN as (pixel - 128)/128. I stack four consecutive observations for motion information and repeat the initial frame at reset. I retain v5's default four-frame action repeat and sticky-action probability .25 [1]; I do not add a second action-repeat wrapper. I apply gamma per agent decision step.

How I separate training, selection and testing

I train with seed 42. I validate CartPole on 20 greedy episodes every 25 training episodes and MsPacman on ten every 100, using seeds starting at 200000. I freeze the best validation checkpoint before 500 greedy tests on seeds 100000-100499. I evaluate uniform-random policies on the same test seeds, without optimization or replay updates. I report sample standard deviations with ddof = 1 and keep all test seeds separate from checkpoint selection.

2. My CartPole training results

I trained CartPole for 1,675 episodes and 226,837 decision steps on CUDA. I observed mean rewards of 21.78 in my first 100 episodes and 254.86 in my last 100. I retain exploration in the training curve, so I assess the selected greedy policy separately.



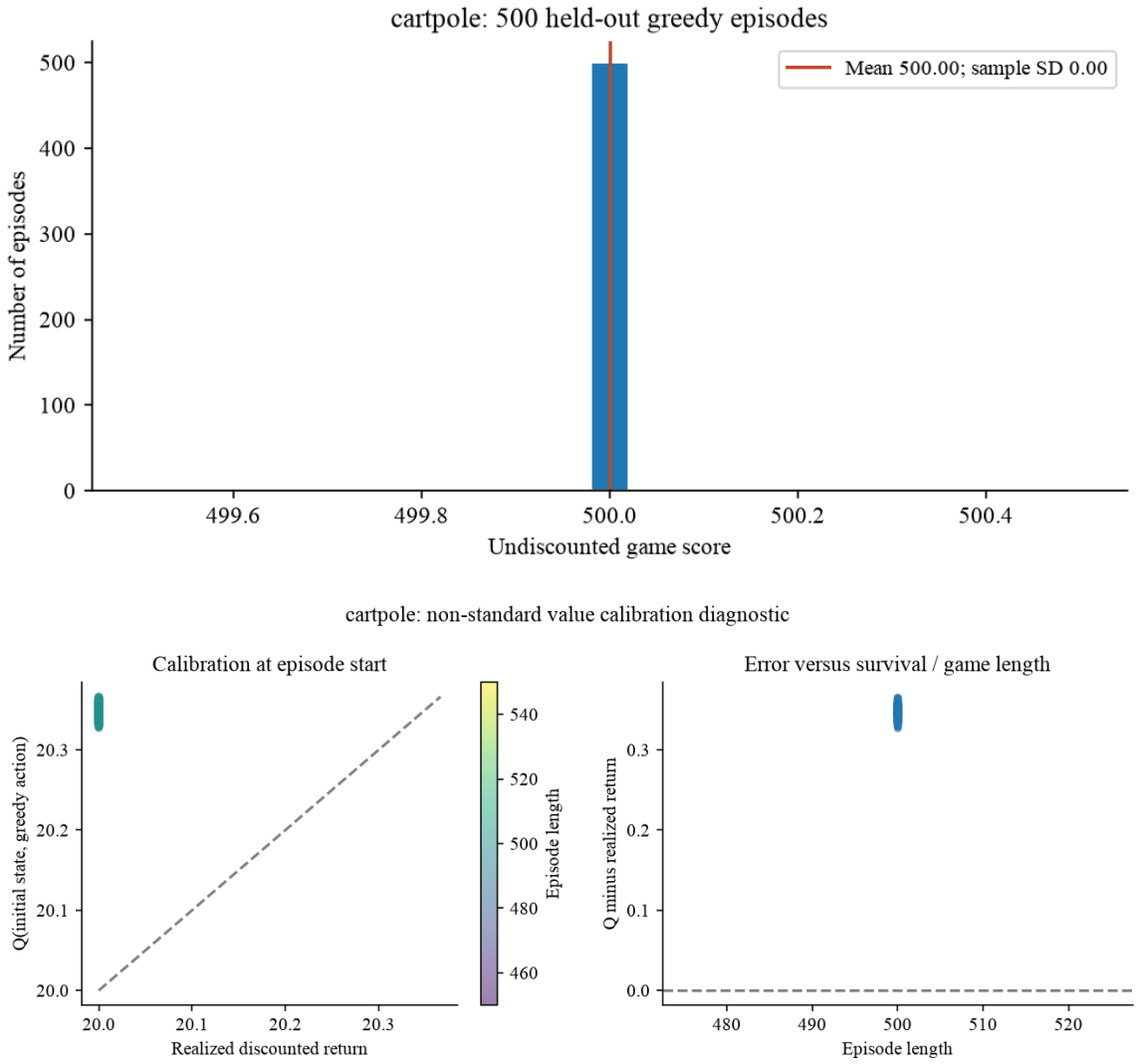
Figures 1-2. I plot maximum action values over visited states and a fixed 128-observation probe, followed by raw rewards and the exact trailing 100-episode mean. I omit the moving average until 100 episodes exist. I use the fixed probe to separate value changes from changes in the visited-state distribution.

I expect constant-reward CartPole values to approach 1/(1-.95) = 20. I therefore do not treat a flat or high Q curve as proof of 500-step control. I use held-out survival and state interventions to check behavior despite non-monotone bootstrapped training.

3. My CartPole held-out evaluation

Policy	Episodes	Mean reward	Sample SD	500-step fraction
My greedy DQN	500	500.000	0.000	100.0%
Uniform random	500	22.020	10.297	0.0%

I measured a paired improvement of 477.980 steps over random (approximate 95% interval 477.077 to 478.883). I interpret this interval as episode variability for one trained model, rather than uncertainty across training seeds. I reached 500 steps in every held-out episode; I do not claim this guarantees success for every possible state.



Figures 3-4. I show the reward histogram and my non-standard value-calibration diagnostic. I compare initial greedy Q with realized discounted return and plot the error against episode length. I measured mean initial Q 20.3472, mean realized return 20.0000, and mean error 0.3472. For these 500-step trajectories, I bound the omitted constant-reward tail by .95^500/.05, about 1.46e-10; that tail cannot explain my larger calibration gap.

4. How I validate learning and edge cases

My experiments with exact predictions

I isolate temporal credit assignment with a deterministic two-state MDP. I let a0 move s0 to s1 for zero reward and then terminate from s1 with reward c; I let a1 quit for zero reward from either state. I expect Q(s0,a0) = gamma*c, Q(s1,a0) = c, and zero quit values. I train through the same update function for 1,500 updates, synchronize the target every 25 updates, and require maximum absolute error below .02.

Gamma	Reward c	My maximum Q error	Prediction I test
0.95	1.0	1.19e-08	Delayed reward
0.99	1.0	9.54e-09	Delayed reward
0.0	1.0	1.49e-08	No delayed value
0.95	3.0	3.34e-07	Triple value scale
0.95	0.0	0	Zero values

I test gamma .95 and .99, remove delayed rewards with gamma 0, triple the reward scale, and set every reward to zero. I recover the predicted values in all cases. I use these counterfactuals to verify discounting and reward propagation beyond format checks; I do not treat the toy linear model as proof of image-based control.

How I handle terminal states, time limits and replay

I stop bootstrapping at physical pole failure or Atari game-over, but retain it at time-limit truncation [2]. I measured targets 1.0 and 10.5 for a controlled terminal/truncated pair with reward 1, gamma .95 and successor value 10. I save the final successor before reset and treat life loss as part of a full Atari game rather than game-over.

I wait through replay warmup before updating. I detach my target network, synchronize it explicitly, and use Huber loss and gradient clipping for outlying TD errors. I reconstruct pixel histories with serial and episode identifiers, pad short histories with the first frame, and exclude overwritten histories. I verified exact numbered-frame histories across resets and ring wrap, and checked that the contrast transform removes RGB sum 448. I leave evaluation scores unclipped.

I also retain the first ten test transitions. In my first CartPole trace I observed actions [1, 1, 1, 1, 1, 1, 0, 0, 0, 0], total reward 10, and no early termination. I provide states, Q values, rewards and termination flags in the trace JSON so I can inspect the interaction step by step. I reproduce the analytic and behavioral checks with validate.py; I document each experiment in docs/REPRODUCING.md.

5. My robustness checks and MsPacman run

I add a controlled pole-angle offset immediately after reset while keeping the other state components and each of 100 held-out seeds fixed. I use this intervention to test recovery outside the narrow training reset distribution without changing the physics or reward.

My angle offset (rad)	Mean return	Sample SD	500-step success
-0.15	500.00	0.00	100.0%
-0.075	500.00	0.00	100.0%
0.0	500.00	0.00	100.0%
0.075	500.00	0.00	100.0%
0.15	500.00	0.00	100.0%

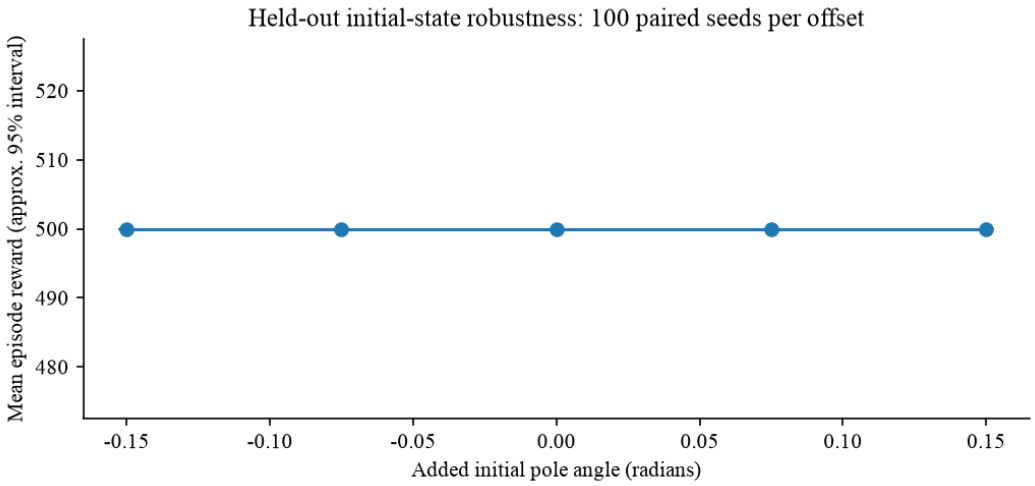
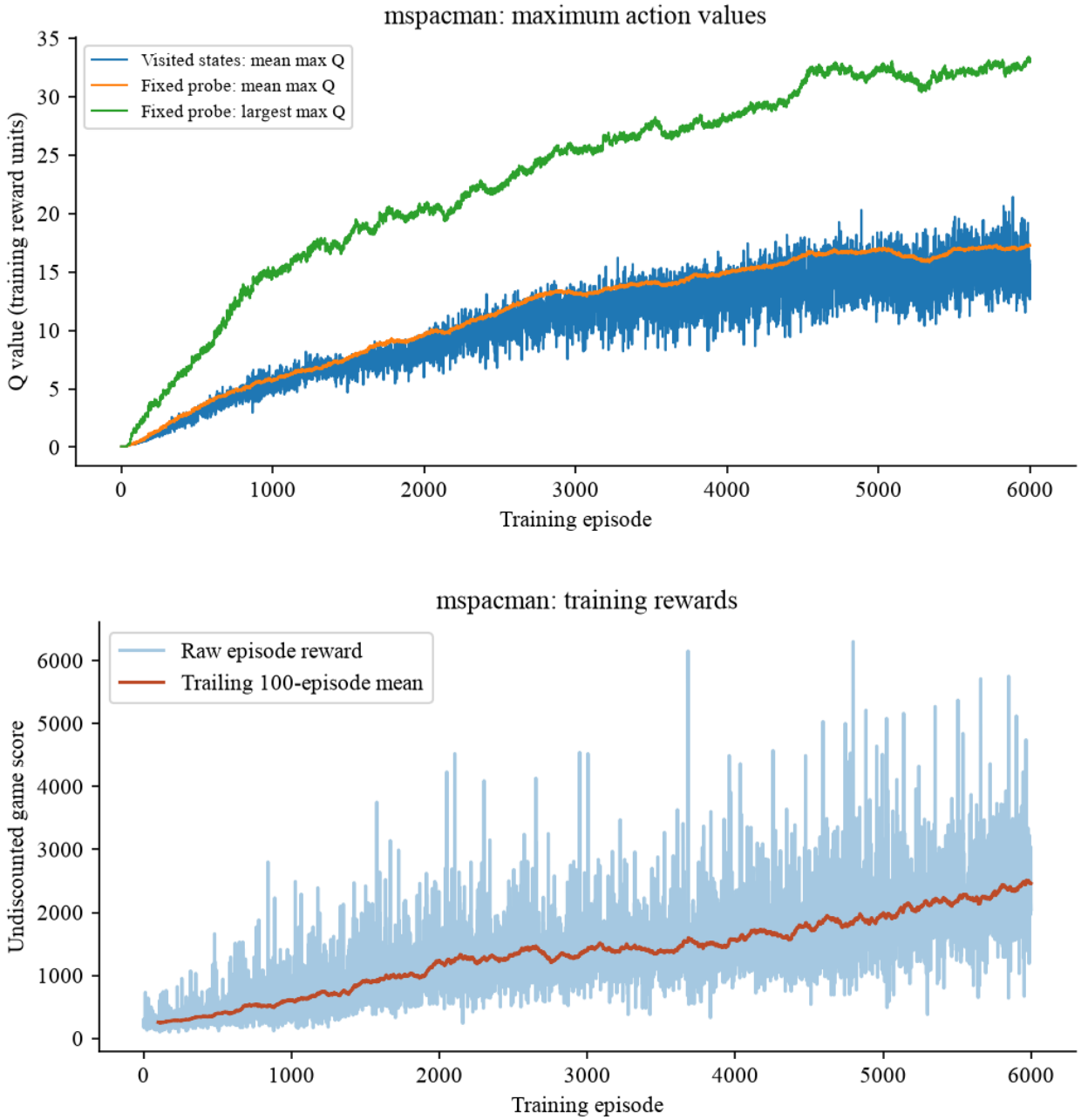


Figure 5. I reached 500 steps in every sampled episode at all five offsets, so my observed intervals collapse to zero width. I interpret this as evidence within the tested range, not as coverage of all state perturbations or variability across training seeds.

My completed MsPacman experiment

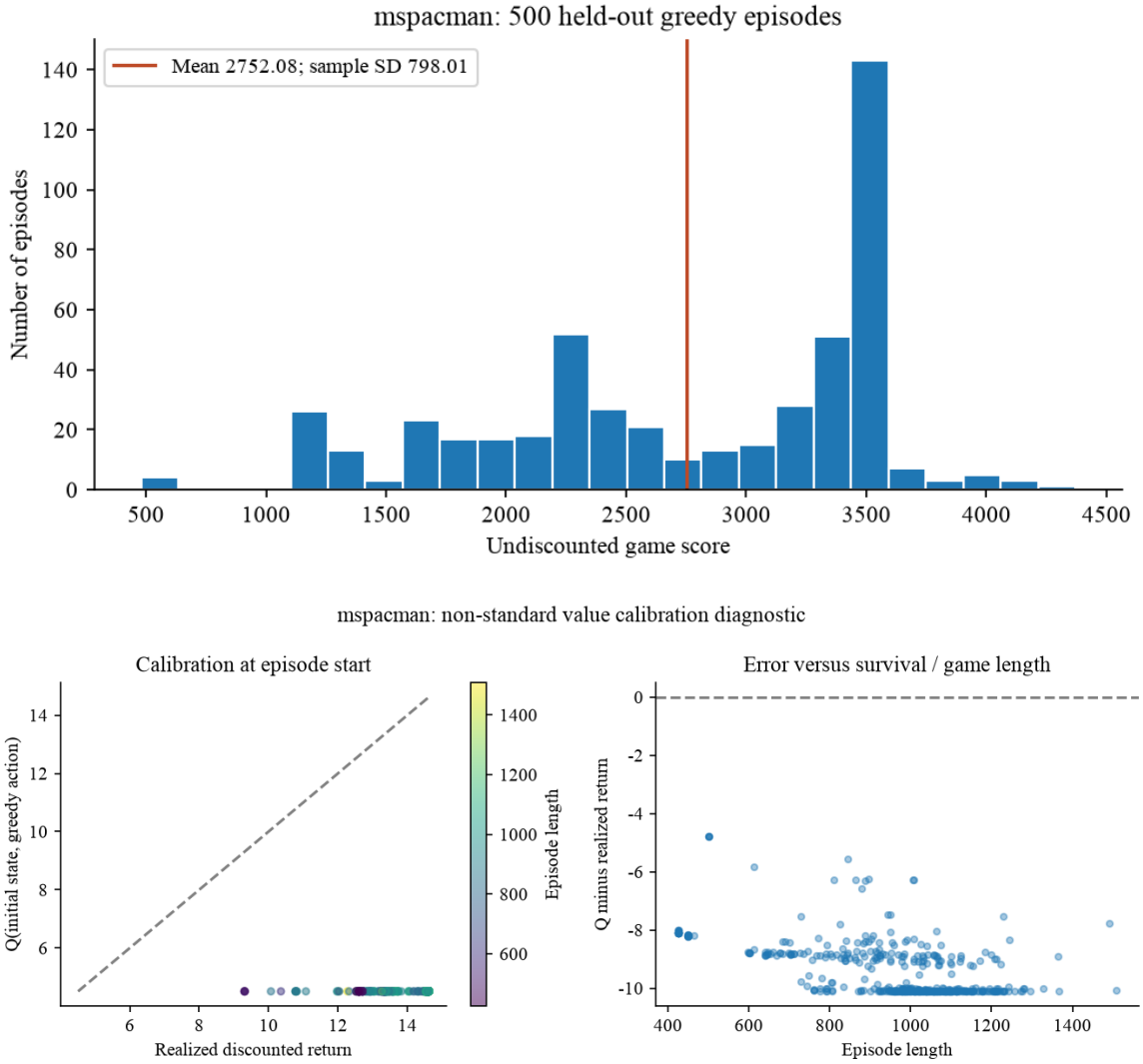
I completed 6,000 MsPacman episodes and 4,428,106 decision steps. I selected episode 5800 using its ten-episode validation mean of 2676.00, before testing. I then measured a 500-episode mean raw score of 2752.08 and sample SD 798.01.

6. My MsPacman learning curves



Figures 6-7. I plot maximum action values in clipped-reward units and raw game scores with the trailing 100-episode mean. I increased my mean raw training score from 255.60 in the first 100 episodes to 2461.50 in the last 100. I interpret this trend as learning in this run; I do not infer full convergence or stability across training seeds from one curve.

7. My MsPacman held-out performance



Figures 8-9. I show my 500-episode raw-score histogram and my non-standard calibration diagnostic in clipped discounted-return units. I measured random mean 256.88 and sample SD 122.59; my paired score improvement is 2495.20, with approximate 95% interval [2424.29, 2566.11]. I condition this interval on the one trained model.

I measured mean initial Q 4.511, versus realized clipped discounted return 13.957, for mean error -9.446. I obtain the same initial image across resets, so my initial Q is constant while later trajectories vary. I observe substantial underprediction despite improved raw scores, which limits my claim about value calibration and convergence. I distinguish terminal and time-limit trajectories in the raw CSV because finite observed prefixes can censor bootstrapped returns.

8. How I test dependence on image information

I freeze my selected CNN and evaluate 50 paired diagnostic seeds (500000-500049) with intact history, the latest frame repeated four times, and a uniform intensity-128 input. I remove temporal differences in the second condition and all state information in the third while leaving the true environment unchanged. I do not use these seeds to select or retune my policy.

My inference condition	Mean score	Sample SD	Action entropy (bits)
Intact history	2785.80	921.84	3.033
Latest frame repeated	1761.40	834.69	3.070
Constant image	210.00	0.00	0.000

I measured intact minus latest frame repeated: paired mean difference 1024.40, approximate 95% interval [671.75, 1377.05].

I measured intact minus constant image: paired mean difference 2575.80, approximate 95% interval [2320.28, 2831.32].

I predicted that removing useful appearance or motion information would reduce scores. I found both paired intervals above zero in favor of intact history, supporting dependence on image information and temporal differences in my frozen model. I also recovered zero action entropy with the constant image, as I expect from a deterministic policy receiving identical inputs at every step.

I treat these interventions as distribution shifts, not equal-budget retraining ablations. I therefore do not claim that they prove optimal control or isolate every visual feature. I retain the raw diagnostic scores in results/mspacman/ablation.csv and the exact checkpoint SHA-256 in validation.json. I reproduce the checks with python validate.py --checkpoint results/cartpole/best.pt --pixel-checkpoint results/mspacman/best.pt --pixel-episodes 50 --out results/validation.json. I keep the required 500-episode test separate from these 50-seed diagnostics.

What I would do with three additional months

Month 1: I would test reproducibility and causality

I would train at least five independent seeds per environment under fixed budgets in environment frames and GPU hours, reserving new final seeds before tuning. I would compare DQN, Double DQN, no target network and no replay with equal budgets, recording scores, TD errors, target drift and intervals. I would test whether removing stabilization actually worsens behavior. I would extend my analytic control with a tiny deterministic image MDP and a zero-reward control to verify the CNN/replay path against known values.

Month 2: I would test coverage and reward design

I would sweep CartPole angle, angular velocity, cart position and cart velocity on a predefined grid, reporting return and failure probability by region. I would vary mass and force within documented ranges as robustness tests while retaining the original graded task for final results. For MsPacman, I would retrain one-frame and four-frame models, compare sign clipping with scaled raw rewards, and vary replay capacity under equal frame budgets. I would inspect ghost/fruit encounters, life loss, action repetition and stagnation using trajectories and event logs.

Month 3: I would test calibration, stress and statistics

I would archive fixed states and collect repeated greedy rollouts with independent environment randomness to estimate return distributions against Q. I would separate terminal and censored trajectories, bootstrap paired score improvements, and aggregate across training seeds. I would predeclare stopping rules and minimum meaningful gains, then evaluate once on untouched final seeds. I would stress interrupted jobs, GPU determinism, memory limits, replay overwrite and long games, implementing exact replay/optimizer/RNG checkpoints before relying on resumed training.

My submission and reproducibility record

I include my public repository identity, all source, raw evidence, required figures for both environments, 500-episode statistics, validation discussion and deepdish single-GPU provenance. I provide this PDF for Canvas submission and keep the complete report preview in README.md. I preserve the experimental limitations rather than treating successful policy evaluation as proof of perfect Q calibration.

References

[1] Farama Foundation. MsPacman, Arcade Learning Environment. https://ale.farama.org/environments/ms_pacman/

[2] Farama Foundation. Handling Time Limits. https://gymnasium.farama.org/tutorials/gymnasium_basics/handling_time_limits/

[3] Mnih et al. Human-level control through deep reinforcement learning. Nature 518, 529-533 (2015). https://www.nature.com/articles/nature14236